

LLM-Augmented Retrieval: Enhancing Retrieval Models Through Language Models and Doc-Level Embedding

Mingrui Wu
Meta
mingruiwu@meta.com

Sheng Cao
Meta
rickcao@meta.com

Abstract

Recently embedding-based retrieval or dense retrieval have shown state of the art results, compared with traditional sparse or bag-of-words based approaches. This paper introduces a model-agnostic doc-level embedding framework through large language model (LLM) augmentation. In addition, it also improves some important components in the retrieval model training process, such as negative sampling, loss function, etc. By implementing this LLM-augmented retrieval framework, we have been able to significantly improve the effectiveness of widely-used retriever models such as Bi-encoders (Contriever, DRAGON) and late-interaction models (ColBERTv2), thereby achieving state-of-the-art results on LoTTE datasets and BEIR datasets.

1 Introduction

The Bi-encoder (Karpukhin et al., 2020) is a type of neural network architecture that is widely used in information retrieval. It consists of two encoders, typically in the form of transformer models (Vaswani et al., 2017), which **encode an vector representation for user queries and potential documents or passages respectively**. These two encoders can be shared or using two separate models. The similarity between these two embedding vectors can then be computed, often using dot product or cosine similarity, to determine the relevance of the document or passage to the user’s query.

Cross-encoders (Nogueira and Cho, 2019), unlike bi-encoders, amalgamate the inputs at an early stage, allowing for a more intricate interaction between user queries and documents. Here the user query and the document are concatenated, based on which a joint embedding vector is computed. The joint embedding vector is then used to make predictions, such as the relevance of a document to a query in an information retrieval task. Cross-encoders often outperform bi-encoders in tasks re-

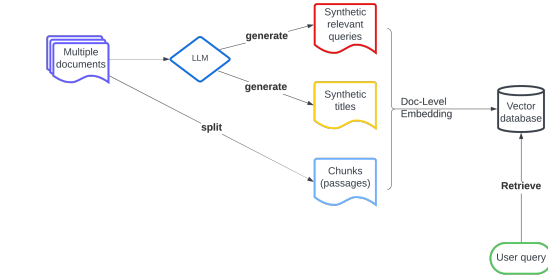


Figure 1: Overall view on LLM-augmented retrieval framework. **Synthetic relevant queries and synthetic titles are generated from LLM and then assembled into doc-level embedding together with chunks (passages) split from the original document. The final retrieval is based on the similarity between user query and the doc-level embedding.**

quiring a nuanced understanding of the interplay between inputs.

Late-interaction models, such as ColBERT (Khattab and Zaharia, 2020), ColBERTv2 (Santhanam et al., 2021) or SPALDE++ (Formal et al., 2022), are model architectures that hybrids cross-encoder models and bi-encoder models. Queries and documents are independently encoded into token-level vector representations. So in some sense, this is a bag of embedding vectors model. The interaction between these representations, which constitutes the “late interaction”, involves computing the cosine similarity or dot product scores over the token-level vector embedding.

All the model architectures require informative embedding of user queries and target documents. While we cannot control the user queries during retrieval tasks, we hypothesize that enriching the embedding of documents can improve the quality and robustness of text retrieval.

This work makes the following contributions:

1. We propose LLM-augmented retrieval, a model-agnostic framework that enriches the

contextual information in the vector embedding of documents to improve the quality and robustness of existing retrievers.

2. We propose doc-level embedding, which combines more contextual information in the context embedding.
3. We evaluate this framework across different models and a wide range of datasets, establishing state-of-art quality beyond original models.
4. We propose improvements to some key components in retrieval model training, such as negative sampling, loss function, etc.

2 Background & Related Work

To the best of our knowledge, there’s no existing work on constructing a unified LLM-augmented framework for doc-level embedding. Therefore, we summarized some related knowledge that’s important to our work.

2.1 Data Augmentation in Information Retrieval

Data augmentation is a widely used technique in information retrieval training. In Contrastive Learning (Izacard et al., 2021), inverse cloze task, independent cropping, as well as random word deletion, replacement or masking are introduced to enrich the diversity of training data. While in training the DRAGON model, (Lin et al., 2023), not only query augmentation by query generation models are studied, but also label augmentation methods with diverse supervision are discussed and compared.

Large pre-trained language models are good at generating text data of high quality (Anaby-Tavor et al., 2020; Papanikolaou and Pierleoni, 2020; Yang et al., 2020; Kumar et al., 2020; Schick and Schütze, 2021; Meng et al., 2022). Some past work has been done to utilize the generation capability of language models to create synthetic training data for retriever models (Bonifacio et al., 2022; Wang et al., 2023).

2.2 Pseudo Queries Generation

Pre-generated pseudo queries are proved to be effective in improving retrieval performance. Previously the similarity of pseudo-queries and user-queries are calculated through BM25 or BERT model in determining the final relevance score of

the query to document through relevance score fusion (Chen et al., 2021; Wen et al., 2023). An alternative method to generate pseudo queries is to generate pseudo query embedding through K-means clustering algorithm (Tang et al., 2021).

2.3 Retrieval Augmented Generation (RAG)

In a Retrieval Augmented Generation (RAG) system (Lewis et al., 2020), a dedicated retriever module is introduced to retrieve relevant documents from a set of corpus based on the input query, and the retrieved documents are then integrated by a language model as part of the context, in order to refine its final response generation. This approach can be applied to not only decoder-only models but also encoder-decoder architectures (Yu, 2022).

2.4 Similarity Scores

Having the embedding vectors of input query and target document, we need to compute the similarity score between these two vectors to determine the relevance between them (Jones and Furnas, 1987). Dense retrievers like Contriever (Izacard et al., 2021) and DRAGON (Lin et al., 2023) use dot products to measure the similarity of query and document in the text embedding space. SPLADE (Formal et al., 2021b) and SPLADEv2 (Formal et al., 2021a) use dot products as well while ColBERT (Khattab and Zaharia, 2020) and ColBERTv2 (Santhanam et al., 2021) use cosine similarity scores. The main difference between cosine similarity and dot product is the sensitivity to the magnitude of the embedding vectors. And dot product is computationally cheaper and more stable.

2.5 Training

The contrastive InfoNCE loss (Izacard et al., 2021; Lin et al., 2023) is widely adopted in dense retrieval training and is defined as:

$$L(q, d_+) = -\frac{\exp(s(q, d_+)/\tau)}{\exp(s(q, d_+)/\tau) + \sum_{i=1}^K \exp(s(q, d_i)/\tau)} \quad (1)$$

where τ is the temperature, s is the similarity score function, q is the input query, d_i is any candidate document and d_+ is the relevant document. Other popular loss functions include point-wise Binary Cross Entropy (Sun et al., 2023), list-wise Cross Entropy (Bruch, 2021), RankNet (Burgess et al., 2005) and LambdaLoss (Wang et al., 2018). Pointwise Binary Cross Entropy is calculated based on each of the query-document pairs independently. List-wise Cross-Entropy loss is widely used in passage

ranking and minimizes list-wise softmax cross entropy on all passages. RankNet is used to calculate the pair-wise relevant order of passages.

3 LLM-augmented Retrieval

In this section, we first discuss the components of the LLM-augmented retrieval framework. After that, we explain how this framework can be adapted to different retriever model architectures. In particular, we propose doc-level embedding for bi-encoders and late-interaction encoders under the LLM-augmented retrieval framework and demonstrate how it is applied to enhance the end-to-end retrieval quality.

3.1 LLM-Augmented Retrieval Framework

3.1.1 Synthetic Relevant Queries

The inspiration for this concept was drawn from web search techniques (Xue et al., 2004; Guo et al., 2009a,b; Chuklin et al., 2022). To illustrate the idea, let us consider an example on a user query of "MIT". Without prior knowledge, it is challenging to figure out that "Massachusetts Institute of Technology" and "MIT" are equivalent. Nevertheless, in web search, we can observe that the home page of "Massachusetts Institute of Technology" has received numerous clicks from the query "MIT", allowing us to infer that the home page of "Massachusetts Institute of Technology" must be closely associated with the query "MIT". On the other side, we typically don't have click data for each user query in the scenario of contextual retrieval. However, large language models are good at generating synthetic queries (Anaby-Tavor et al., 2020; Papanikolaou and Pierleoni, 2020; Yang et al., 2020; Kumar et al., 2020; Schick and Schütze, 2021; Meng et al., 2022) so we can use the synthetic queries as proxy "click data" to direct user queries to related documents.

An important point is that in traditional retrieval tasks, we are using similarity to express relevance (Jones and Furnas, 1987). The similarity score is mathematically defined as either dot product or cosine of the encoded vectors of user query and documents. However, sometimes this similarity score might not reflect semantic relevance (Rao et al., 2019). For example, "Who is the first president of the United States?" might be very close, in terms of similarity score, to "Who became the first president of America?". But our target answer might be a Wiki page or autobiography about "George

Washington", whose similarity score to the query is not that high. While if we use Washington's autobiography to create synthetic queries, "Who became the first president of America?" may be one of them. The user query "Who is the first president of the United States?" can easily match to the relevant query through similarity scores. And the latter points to the target document (Washington's autobiography). Therefore **the generated synthetic queries express the semantic of the original document from different angles which are helpful to match the relevant queries.**

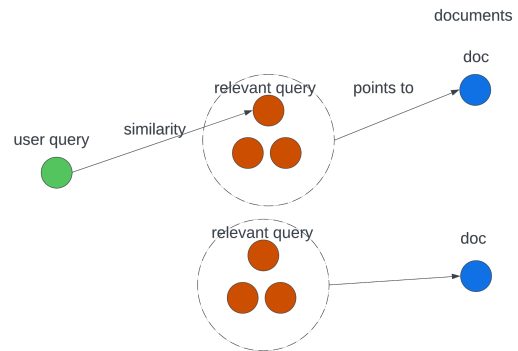


Figure 2: Through synthetic relevant queries, the relevance relationship is not solely expressed by the similarity now but also expressed by the augmentation steps of the large language models

3.1.2 Title

The title of a document plays a crucial role in determining its relevance and usefulness to a user's query. The title is often the first piece of information that a user sees when searching for documents, and it can greatly influence their decision to click on a particular result. A well-crafted title can provide important context and keywords that help users quickly understand the content and purpose of a document. If the original document has a title, we can use them directly. If it does not, we can leverage the large language models to generate a synthetic title for a that document.

3.1.3 Chunks (Passages)

Chunking refers to the process of dividing a large document or piece of text into smaller, more manageable units. These units, known as "chunks," or "passages", are typically created by grouping together related pieces of information. Due to the limitation of the context window of retriever models, (in other words, max length of the model input),

we typically divide a long document into several chunks whose number of tokens is **below the context window limit**. The chunk data is from original documents and is not from LLM-augmentation. **The optimal chunking size is different for various retriever models**. For bi-encoders like Contriever and DRAGON, we found the optimal chunking size to be 64 after empirical studies. For token-level late-interaction models such as ColBERT and ColBERTv2, since it's already calculating the similarity score at token-level, it's not necessary to chunk the original documents unless the context window limit is reached.

3.2 Doc-level embedding

In this section, we will first introduce the high level idea of doc-level embedding for information retrieval, and then use bi-encoders and token-level late-interaction models to illustrate how the doc-level embedding can be adaptive to different retriever model structures.

Document fields. For convenience, we call the above mentioned information source, synthetic queries, title and chunk, *fields* of a document. These **fields express the semantic of the original document from different angles and will be composed into the doc-level embedding of a document** which is static and can be pre-computed and cached for information retrieval. Embedding indexes can be pre-built to speed up the retrieval inference, and each doc-level embedding points to the original document.

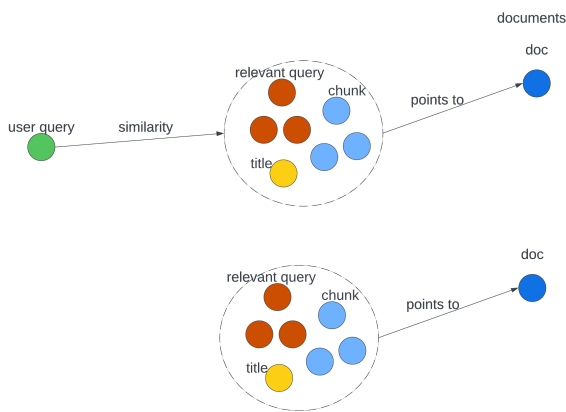


Figure 3: The graphic representation of "relevance" in doc-level embedding

3.2.1 For Bi-encoders

Bi-encoders are typically **"Two-Tower" model** structures. Given a query and a document, a query

encoder and a doc encoder are applied to **compute the embedding** vectors for the query and the document respectively. Then these two embedding vectors are fed into the **dot products** (or cosine similarity) to **compute the similarity scores between the query and the document**. As we want to enrich the document embedding vectors by injecting the synthetic queries and titles, we propose to compute the similarity as the following:

$$\text{sim}(q, d) = \max_i s(q, c_i) + \sum_f w_f \times s(q, e_f) \quad (2)$$

The first term on the right hand side calculates the maximum similarity score of query chunk embedding pairs, where s is the similarity score function, q is the embedding vector of the input query, c_i is the embedding vector of the i -th chunk in the document. This term is commonly used in **current embedding based retrievals**, which determines the similarity between a query and a document based on the query and the most relevant chunk in the document.

The second term is innovative and considers more information, where e_f is the embedding vector of each document field. The **similarity scores between the query embedding and each field embedding are computed and combined together**, each with a field weight parameter w_f . As described before, those document fields include the synthetic query, title and chunk fields.

Now let's consider how to compute the query field embedding e_f for each field. For the title field, which just contains one phrase or sentence, it is straightforward. We just apply the doc encoder to compute the embedding vector of the title as the title field embedding.

For the **chunk field** which may contain multiple doc chunks, we can **compute the embedding vector of each chunk with the doc encoder**. The question is how to combine these embedding vectors to represent the whole document? Actually people have already considered a similar problem for sentence embedding: having the embedding vectors of all the tokens in one sentence, how to come up with a representation of the whole sentence? A simple but effective idea was proposed in (Arora et al., 2017) for this problem. Here we adapt this idea to the doc embedding problem. Namely we **compute the average of all the chunk embedding vectors as the chunk field embedding**. Similarly, for the synthetic query field, we compute the embedding vector of each query with the query encoder, and then **compute the average of these embedding vectors as**

the query field embedding. This simple approach works very well in our experiments, while clearly more advanced approaches can be explored here in the future.

Furthermore, as the similarity function is linear¹, the above equation can be simplified as the following:

$$\text{sim}(q, d) = \max_i s(q, c_i + \sum_f w_f \times e_f) \quad (3)$$

Therefore, we can treat $c_i + \sum_f w_f \times e_f$ as the chunk embedding vector of each chunk c_i of the original document, and still apply the algorithms like approximate nearest neighbors (Indyk and Motwani, 1998) to retrieve the most relevant documents.

3.2.2 For Token-Level Late-Interaction Models

Instead of using a single embedding vector for query and a single embedding vector for each document, late-interaction models such as ColBERT and ColBERTv2 use token-level embedding and the embedding vectors of all the tokens are kept and will participate in computing the similarity score between the query and document.

$$\text{sim}(q, d) = \sum_i \max_j s(q_i, t_j) \quad (4)$$

where q_i and t_j are token-level embedding vectors for the input query and document respectively. Therefore, for each query token, the most similar token from the document is identified, and their similarity score is recorded. All these scores are summed up over all the query tokens to obtain the overall similarity between the query and the document. Since the similarity score calculation is done at token-level, we can concatenate the synthetic queries and titles to the original document passages. After that, we decide whether to chunk the concatenated documents if the number of tokens reaches the context window limit.

4 Experiments

In this section we will introduce our experiment set up, the datasets and the models used in our experiments.

¹Dot product is linear. Cosine similarity is also linear if we normalize the embedding vectors to unit length.

4.1 Datasets

BEIR Data The BEIR (Benchmark for Evaluating Information Retrieval) dataset (Thakur et al., 2021), provides a comprehensive benchmark for evaluating and comparing different IR models, particularly in the context of out-of-domain tests. BEIR is designed to address the limitations of other previous datasets by offering a more diverse and extensive range of queries and passages. It encompasses a wide array of topics, thereby facilitating a more robust and comprehensive evaluation of IR models.

LoTTE Data LoTTE data (Santhanam et al., 2021) is a dataset specifically designed for Long-Tail Topic-stratified Evaluation. The primary focus of LoTTE is on natural user queries that are associated with long-tail topics, which may not be adequately covered by an entity-centric knowledge base such as Wikipedia. LoTTE is composed of 10 distinct test sets, each containing 500 – 2,000 queries and 100,000 – 2,000,000 passages. These test sets are divided by topic, and each is accompanied by a validation set comprising related yet disjoint queries and passages. In this experiment we are using the test split solely for evaluation.

4.2 Models

Contriever For Contriever we use the Roberta-base (Liu et al., 2019) model architecture, the checkpoint is trained on Wiki passages (Karpukhin et al., 2020) and CC100 (Conneau et al., 2019) data through Contrastive Learning. It has 125M parameters, a context window of 512 tokens, with 12 layers, 768 hidden dimensions, and 12 attention heads. We use the single Roberta-base model for query encoder and context encoder, namely the “Two Tower” of the bi-encoder is shared in model architecture.

DRAGON For DRAGON we also use the Roberta-base model architecture, the checkpoint is trained by the author and released publicly. Different from the Contriever model, DRAGON has separate Roberta-base models for query encoder and context encoder.

ColBERTv2 For ColBERTv2, we use bert-base-uncased model architecture, the same as the default settings in the original paper. It has 110M parameters and a context window of 256 tokens, with 12

layers, 768 hidden dimensions, and 12 heads. The checkpoint is trained on the MSMARCO dataset (Nguyen et al., 2016) from the author’s checkpoint.

4.3 Implementation Details

We choose open source Llama-70B (Touvron et al., 2023a,b) for synthetic query generation and title generation. The prompt templates used for generating synthetic queries and titles are in Table 9 and 10.

For Bi-encoders, we implemented the doc-level embedding as above mentioned and chose $w_{query}=1.0$, $w_{title}=0.5$, $w_{chunk}=0.1$ for the Contriever model and $w_{query}=0.6$, $w_{title}=0.3$, $w_{chunk}=0.3$ for the DRAGON model. We chose $chunk_size=64$ after empirical experiments and found 64 normally performed the best in retrieval results for Bi-encoders. Note these hyper-parameters of field weights are not fully optimized. They are chosen according to the performance of Bi-encoder models on the single LoTTE-lifestyle-forum dataset, and then fixed for the evaluations on all the other datasets.

For ColBERTv2, as mentioned previously, we concatenate the title with all the synthetic queries for each document and make it an additional “passage” of the original document. Thus there’s no field weights hyper-parameters in these experiments. There could be other better assembling methods for composing the doc-level embedding under a late-interaction model architecture. We set $index_bits=8$ when building the ColBERT index.

5 Results

The result on LoTTE and BEIR data for all three models can be seen in Table 1, 2 and 3. We can observe that the LLM augmented retrieval and doc-level embeddings boost the recall@3 and recall@10 of Bi-encoders (Contriever and DRAGON) significantly. For token-level late-interaction models (ColBERTv2), the increase on the LoTTE and BEIR Dataset is still clear, although not as much as on Bi-encoders. We hypothesis this due to that the baselines for token-level late-interaction models are much higher than those of Bi-encoders.

Moreover, the performance of the LLM-augmented Contriever has exceeded the performance of the vanilla DRAGON in most datasets. Similarly, the LLM-augmented DRAGON even exceeds vanilla ColBERTv2 on BEIR-ArguAna,

BEIR-SciDocs and BEIR-CQADupstack-English datasets and greatly reduce the performance gap in the remaining datasets, although ColBERTv2 introduces a more complex late-interaction architecture than DRAGON. Therefore, we can see that, after enriching the embedding of documents with LLM augmentation, we can greatly improve the retriever models recall performance without further fine-tuning.

6 Ablation Studies

In this section, we want to see how the chunk, query and title fields impact the retrieval quality of different retriever models. For Bi-encoders (Contriever and DRAGON), we further control the field weights of chunk, synthetic query and title to see how those parameters can affect the performance. For the token-level late-interaction model (ColBERTv2), we just control the model to use one of the chunk, query or title field only to see how they affect the end-to-end retrieval quality.

For the Contriever model (Table 4), we have observed that most of the time the synthetic queries play the most critical role in boosting the recall performance, comparing to the other two fields, while in BEIR-SciDocs and BEIR-Scifact, synthetic queries’ importance is smaller. Therefore, a weighted sum of multiple fields in doc-level embedding yields better performance in most cases. And these weights can be further tuned as hyper-parameters.

For the DRAGON model (Table 5), there’s less strong pattern that which field plays a more important role in doc-level embedding. In the LoTTE dataset it’s more driven by the title field. However, the chunk field matters more in datasets of BEIR-ArguAna, BEIR-Quora and BEIR-SciFact. Once again, a weighted sum of multiple document fields in doc-level embedding yields better performance in most cases. For the different patterns we have observed in DRAGON vs Contriever, one reason might be that DRAGON uses separate query and context encoders, while Contriever uses shared query and context encoders in our setup. Thus Contriever is better at identifying similarity instead of relevance and that’s why the synthetic query field has a more significant impact in the Contriever model, since it better transforms similarity to relevance as explained before.

For ColBERTv2 (Table 6), we have observed from the LoTTE dataset that further chunking the

LoTTE Dataset											
Model	Recall	Lifestyle Forum	Lifestyle Search	Recreation Forum	Recreation Search	Science Forum	Science Search	Technology Forum	Technology Search	Writing Forum	Writing Search
Contriever	recall@3	0.4366	0.3358	0.3486	0.1948	0.1046	0.1005	0.1826	0.1242	0.3950	0.2745
	recall@10	0.6149	0.4690	0.4895	0.2857	0.1706	0.1637	0.3174	0.1896	0.5390	0.3950
Contriever*	recall@3	0.6244	0.6021	0.5455	0.4610	0.2395	0.2901	0.3663	0.3557	0.5970	0.5724
	recall@10	0.7622	0.7821	0.6948	0.6320	0.3570	0.4684	0.5494	0.5017	0.7365	0.6919
BEIR Dataset											
Model	Recall	ArguAna	FIQA	Quora	SciDocs	SciFact	CQADupstack English	CQADupstack Physics			
Contriever	recall@3	0.2589	0.1895	0.8654	0.1580	0.5410	0.2261	0.1723			
	recall@10	0.5206	0.2993	0.9463	0.2950	0.6934	0.3089	0.2551			
Contriever*	recall@3	0.2468	0.3690	0.8687	0.2440	0.5996	0.3822	0.3417			
	recall@10	0.5825	0.5174	0.9517	0.4030	0.7259	0.5025	0.4658			

Table 1: Results on Contriever: The performance of LLM-augmented Contriever has greatly exceeded the vanilla Contriever on both LoTTE and BEIR dataset, and even exceeds the performance of the vanilla DRAGON in most datasets. * means base model plus the doc-level embedding ($w_{query}=1.0$, $w_{title}=0.5$, $w_{chunk}=0.1$).

LoTTE Dataset											
Model	Recall	Lifestyle Forum	Lifestyle Search	Recreation Forum	Recreation Search	Science Forum	Science Search	Technology Forum	Technology Search	Writing Forum	Writing Search
DRAGON	recall@3	0.5270	0.5598	0.4560	0.4253	0.2578	0.2601	0.2854	0.3591	0.5300	0.5798
	recall@10	0.6798	0.7035	0.5949	0.5325	0.3704	0.3938	0.4232	0.5101	0.6675	0.7311
DRAGON*	recall@3	0.6883	0.7625	0.6079	0.6472	0.3099	0.4498	0.4192	0.5285	0.6520	0.7031
	recall@10	0.8172	0.8911	0.7468	0.7944	0.4427	0.6062	0.6038	0.7097	0.7725	0.8170

BEIR Dataset										
Model	Recall	ArguAna	FIQA	Quora	SciDocs	SciFact	CQADupstack English	CQADupstack Physics		
DRAGON	recall@3	0.1408	0.3327	0.8465	0.1800	0.4743	0.2605	0.1877		
	recall@10	0.4040	0.4514	0.9419	0.3260	0.5996	0.3599	0.2916		
DRAGON*	recall@3	0.3663	0.4255	0.8638	0.3040	0.6610	0.4618	0.3936		
	recall@10	0.6764	0.5635	0.9527	0.4800	0.7710	0.5662	0.5342		

Table 2: Results on DRAGON: The performance of LLM-augmented DRAGON has greatly exceeded the vanilla DRAGON on both LoTTE and BEIR dataset, and even exceeds vanilla ColBERTv2 on BEIR-ArguAna, BEIR-SciDocs and BEIR-CQADupstack-English datasets, as well as greatly reduces the performance gap in the remaining datasets. * means base model plus the doc-level embedding ($w_{query}=0.6$, $w_{title}=0.3$, $w_{chunk}=0.3$).

LoTTE Dataset											
Model	Recall	Lifestyle Forum	Lifestyle Search	Recreation Forum	Recreation Search	Science Forum	Science Search	Technology Forum	Technology Search	Writing Forum	Writing Search
ColBERTv2	recall@3	0.6988	0.7927	0.6344	0.6677	0.3932	0.5073	0.4496	0.5940	0.6960	0.7423
	recall@10	0.8087	0.8911	0.7498	0.7868	0.5285	0.6613	0.6292	0.7315	0.8050	0.8366
ColBERTv2*	recall@3	0.7308	0.8003	0.6753	0.7100	0.4026	0.5024	0.4626	0.5956	0.7145	0.7544
	recall@10	0.8447	0.9107	0.7862	0.8268	0.5558	0.6726	0.6517	0.7383	0.8260	0.8571

BEIR Dataset										
Model	Recall	ArguAna	FIQA	Quora	SciDocs	SciFact	CQADupstack English	CQADupstack Physics		
ColBERTv2	recall@3	0.3542	0.4469	0.9048	0.2990	0.6691	0.4484	0.4052		
	recall@10	0.6287	0.5787	0.9643	0.4780	0.7755	0.5369	0.5380		
ColBERTv2*	recall@3	0.3592	0.4666	0.9067	0.3000	0.6862	0.4822	0.4196		
	recall@10	0.6344	0.6018	0.9663	0.4850	0.7917	0.5694	0.5611		

Table 3: Results on ColBERTv2: The performance of LLM-augmented ColBERTv2 has greatly exceeded the performance of vanilla ColBERTv2 on both LoTTE and BEIR dataset. * means base model plus the doc-level embedding.

passages actually hurts the performance. This may be because the similarity calculation is done at token-level so chunking the passages (meanwhile increasing the number of chunks) will not help the ColBERTv2 model digest the granular contextual information. Therefore we did not evaluate the

chunk-only scenario on the BEIR dataset. The synthetic queries are more critical than titles for ColBERTv2 across the datasets, while combining them all often yields even better recall results. Note again there’s no field weights hyper-parameters for token-level late-interaction models.

7 Supervised Fine-tuning

The above studies are all based on zero-shot evaluation, namely the checkpoints in (Lin et al., 2023; Santhanam et al., 2021) are directly used in evaluation. Actually our LLM-augmented retrieval and doc-level embedding also support supervised fine-tuning on domain-specific datasets. Some popular training methods include picking hard negatives, constructing in-batch or cross-batch negatives, and calculating InfoNCE loss as training loss (Izacard et al., 2021; Lin et al., 2023). In this section, we propose some effective training techniques for fine-tuning which we used internally on proprietary dataset and performed better than the common methods mentioned above.

7.1 Adaptive Negative Sampling

The process of acquiring negative docs for each query can be approached in several ways. A straightforward method involves the random sampling of doc contexts, excluding the ground truth. Alternatively, a model such as BM25 (Robertson et al., 2004, 2009) could be employed to select “strong” negative doc contexts that have been assigned high ranking scores by the model. However, for more effective training, it is advantageous to obtain “difficult” negative doc contexts. To this end, an adaptive negative sampling approach was developed. The retrieval model under training is used to rank all doc contexts for each query. The doc contexts ranked in the top positions, excluding the ground truth doc contexts, are selected and then represented as the most challenging negative examples that could potentially confound our retrieval model. Consequently, it is imperative to train the retrieval model to effectively handle these samples.

A significant challenge of this approach is the time and space complexity. For each query (or batch of queries), it is necessary to re-compute the embedding for all doc contexts to identify the top negative examples. These doc context embedding cannot be pre-calculated due to the continuous updating of the model parameters during training, which correspondingly alters the doc context embedding. To address this issue, we can periodically pre-calculate and update the top negative samples for multiple training batches simultaneously.

7.2 Loss Function

In supervised training, for each training query, each doc has a binary label: relevant or not relevant.

In the context of binary label problems, the cross-entropy loss function is a prevalent choice. However, the cross-entropy is primarily for classification problems. This is due to its requirement for model output scores to be maximized for positive examples and minimized for negative examples. Within the realm of information retrieval, the primary challenge lies in the task of ranking of target documents. The absolute numerical values of output scores are not the primary focus. Instead, the focus is on the relative order and gaps between the model outputs of positive and negative doc contexts. Consequently, we adopted the margin ranking loss function (Nayyeri et al., 2019), which aims to maximize the “margin” between the scores of positive and negative examples.

$$Loss(o_1, o_2, y) = \max(0, -y(o_1 - o_2) + \text{margin}) \quad (5)$$

In the above equation, o_1, o_2 are model output scores, the value of y equals 1 or -1, indicating whether o_1 should be larger or smaller than o_2 respectively. For example, when $y=1$, and the gap between o_1 and o_2 is larger than the “margin” parameter, the loss value is 0. Otherwise, the smaller the gap between o_1 and o_2 , the larger the loss value. Our experiments showed that this margin loss was better than the cross entropy loss on proprietary domain data fine-tuning.

8 Conclusion

This paper presents a novel framework, LLM-augmented retrieval, which significantly improves the performance of existing retriever models by enriching the embedding of documents through large language model augmentation. The proposed framework includes a doc-level embedding that encodes contextual information from synthetic queries, titles, and chunks, which can be adapted to various retriever model architectures. The proposed approach has achieved state-of-the-art results across different models and datasets, demonstrating its effectiveness in enhancing the quality and robustness of neural information retrieval. Future research could explore further enhancements to the LLM-augmented retrieval framework, such as the integration of additional contextual information into doc-level embedding, the application of more advanced similarity score measures, more complicated approaches to combine the embedding of multiple chunks/queries into one chunk/query field embedding, etc.

9 Limitations

One limitation of study include the extra computation resources it required in augmenting relevant queries and titles for original documents and sometimes to size of augmented texts can be comparable to the size of the original documents. This computational limitation may restrict the usage of this approach where computational resource is limited.

Another limitation or risk is that the hallucination in large language models may pose extra inaccuracy in augmented corpus to the original documents. Hallucination remains an unsolved problem in the field of large language model's study.

References

- Ateret Anaby-Tavor, Boaz Carmeli, Esther Goldbraich, Amir Kantor, George Kour, Segev Shlomov, Naama Tepper, and Naama Zwerdling. 2020. Do not have enough data? deep learning to the rescue! In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7383–7390.
- Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. A simple but tough-to-beat baseline for sentence embeddings. *International Conference on Learning Representations*.
- Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. 2022. Inpars: Data augmentation for information retrieval using large language models. *arXiv preprint arXiv:2202.05144*.
- Sebastian Bruch. 2021. An alternative cross entropy loss for learning-to-rank. In *Proceedings of the web conference 2021*, pages 118–126.
- Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. [Learning to rank using gradient descent](#). In *Proceedings of the 22nd International Conference on Machine Learning, ICML '05*, page 89–96, New York, NY, USA. Association for Computing Machinery.
- Xuanang Chen, Ben He, Kai Hui, Yiran Wang, Le Sun, and Yingfei Sun. 2021. Contextualized offline relevance weighting for efficient and effective neural retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1617–1621.
- Aleksandr Chuklin, Ilya Markov, and Maarten De Rijke. 2022. *Click models for web search*. Springer Nature.
- Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Unsupervised cross-lingual representation learning at scale. *arXiv preprint arXiv:1911.02116*.
- Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2021a. Splade v2: Sparse lexical and expansion model for information retrieval. *arXiv preprint arXiv:2109.10086*.
- Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2022. From distillation to hard negative sampling: Making sparse neural ir models more effective. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2353–2359.
- Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021b. Splade: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292.
- Fan Guo, Chao Liu, Anitha Kannan, Tom Minka, Michael Taylor, Yi-Min Wang, and Christos Faloutsos. 2009a. Click chain model in web search. In *Proceedings of the 18th international conference on World wide web*, pages 11–20.
- Fan Guo, Chao Liu, and Yi Min Wang. 2009b. Efficient multiple-click models in web search. In *Proceedings of the second acm international conference on web search and data mining*, pages 124–131.
- Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*, pages 604–613.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*.
- William P Jones and George W Furnas. 1987. Pictures of relevance: A geometric analysis of similarity measures. *Journal of the American society for information science*, 38(6):420–442.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*.
- Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 39–48.
- Varun Kumar, Ashutosh Choudhary, and Eunah Cho. 2020. Data augmentation using pre-trained transformer models. *arXiv preprint arXiv:2003.02245*.

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.
- Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2023. How to train your dragon: Diverse augmentation towards generalizable dense retrieval. *arXiv preprint arXiv:2302.07452*.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Yu Meng, Jiaxin Huang, Yu Zhang, and Jiawei Han. 2022. Generating training data with language models: Towards zero-shot language understanding. *Advances in Neural Information Processing Systems*, 35:462–477.
- Mojtaba Nayeri, Xiaotian Zhou, Sahar Vahdati, Hamed Shariat Yazdi, and Jens Lehmann. 2019. Adaptive margin ranking loss for knowledge graph embeddings via a correntropy objective function. *arXiv preprint arXiv:1907.05336*.
- Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. Ms marco: A human generated machine reading comprehension dataset. *choice*, 2640:660.
- Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*.
- Yannis Papanikolaou and Andrea Pierleoni. 2020. Dare: Data augmented relation extraction with gpt-2. *arXiv preprint arXiv:2004.13845*.
- Jinfeng Rao, Linqing Liu, Yi Tay, Wei Yang, Peng Shi, and Jimmy Lin. 2019. Bridging the gap between relevance matching and semantic matching for short text similarity modeling. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 5370–5381.
- Stephen Robertson, Hugo Zaragoza, and Michael Taylor. 2004. Simple bm25 extension to multiple weighted fields. In *Proceedings of the thirteenth ACM international conference on Information and knowledge management*, pages 42–49.
- Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.
- Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2021. Colbertv2: Effective and efficient retrieval via lightweight late interaction. *arXiv preprint arXiv:2112.01488*.
- Timo Schick and Hinrich Schütze. 2021. Generating datasets with pretrained language models. *arXiv preprint arXiv:2104.07540*.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Pengjie Ren, Dawei Yin, and Zhaochun Ren. 2023. Is chatgpt good at search? investigating large language models as re-ranking agent. *arXiv preprint arXiv:2304.09542*.
- Hongyin Tang, Xingwu Sun, Beihong Jin, Jingang Wang, Fuzheng Zhang, and Wei Wu. 2021. Improving document representations by generating pseudo query embeddings for dense retrieval. *arXiv preprint arXiv:2105.03599*.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023. Improving text embeddings with large language models. *arXiv preprint arXiv:2401.00368*.
- Xuanhui Wang, Cheng Li, Nadav Golbandi, Michael Bendersky, and Marc Najork. 2018. The lambdaloss framework for ranking metric optimization. In *Proceedings of the 27th ACM international conference on information and knowledge management*, pages 1313–1322.
- Xueru Wen, Xiaoyang Chen, Xuanang Chen, Ben He, and Le Sun. 2023. Offline pseudo relevance feedback for efficient and effective single-pass dense retrieval. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2209–2214.

Gui-Rong Xue, Hua-Jun Zeng, Zheng Chen, Yong Yu, Wei-Ying Ma, WenSi Xi, and WeiGuo Fan. 2004. Optimizing web search using web click-through data. In *Proceedings of the thirteenth ACM international conference on Information and knowledge management*, pages 118–126.

Yiben Yang, Chaitanya Malaviya, Jared Fernandez, Swabha Swayamdipta, Ronan Le Bras, Ji-Ping Wang, Chandra Bhagavatula, Yejin Choi, and Doug Downey. 2020. Generative data augmentation for common-sense reasoning. *arXiv preprint arXiv:2004.11546*.

Wenhao Yu. 2022. Retrieval-augmented generation across heterogeneous knowledge. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Student Research Workshop*, pages 52–58.

Appendix

Here we share the ablation studies’ results and the prompts we used in generating synthetic queries and titles using Llama-70B.

LoTTE Dataset											
Model	Recall	Lifestyle Forum	Lifestyle Search	Recreation Forum	Recreation Search	Science Forum	Science Search	Technology Forum	Technology Search	Writing Forum	Writing Search
Contriever $w_{chunk}=1.0$	recall@3	0.4476	0.4342	0.3871	0.2695	0.1358	0.1896	0.2116	0.1879	0.4425	0.3968
	recall@10	0.6459	0.6172	0.5415	0.4145	0.2196	0.3063	0.3693	0.3003	0.600	0.5369
Contriever $w_{query}=1.0$	recall@3	0.6194	0.6967	0.5355	0.4437	0.2335	0.2901	0.3523	0.3305	0.5860	0.5472
	recall@10	0.7762	0.7837	0.6863	0.6115	0.3461	0.4295	0.5180	0.4883	0.7410	0.6910
Contriever $w_{title}=1.0$	recall@3	0.5310	0.4902	0.4975	0.3789	0.2345	0.1896	0.3468	0.2668	0.5315	0.4809
	recall@10	0.6958	0.6641	0.6404	0.5314	0.3421	0.3241	0.5200	0.4077	0.6725	0.6153
Contriever*	recall@3	0.6244	0.6021	0.5455	0.4610	0.2395	0.2901	0.3663	0.3557	0.5970	0.5724
	recall@10	0.7622	0.7821	0.6948	0.6320	0.3570	0.4684	0.5494	0.5017	0.7365	0.6919

BEIR Dataset								
Model	Recall	ArguAna	FIQA	Quora	SciDocs	SciFact	CQADupstack English	CQADupstack Physics
Contriever $w_{chunk}=1.0$	recall@3	0.2240	0.2623	0.8653	0.1980	0.6177	0.2605	0.2156
	recall@10	0.5391	0.4031	0.9463	0.3360	0.7466	0.3580	0.3292
Contriever $w_{query}=1.0$	recall@3	0.2347	0.3580	0.8622	0.2180	0.5888	0.3860	0.3330
	recall@10	0.5718	0.5045	0.8088	0.3720	0.7322	0.5013	0.4629
Contriever $w_{title}=1.0$	recall@3	0.2063	0.3180	0.7555	0.2600	0.5573	0.3338	0.2926
	recall@10	0.5192	0.4595	0.8791	0.4120	0.7051	0.4369	0.4100
Contriever*	recall@3	0.2468	0.3690	0.8687	0.2440	0.5996	0.3822	0.3417
	recall@10	0.5825	0.5174	0.9517	0.4030	0.7259	0.5025	0.4658

Table 4: Ablation study on doc-level embedding with Contriever. In most cases the ensemble of relevant queries, title and chunks gives the best results. * means base model plus the doc-level embedding (chunk:0.1, query:1.0, title:0.5).

LoTTE Dataset											
Model	Recall	Lifestyle Forum	Lifestyle Search	Recreation Forum	Recreation Search	Science Forum	Science Search	Technology Forum	Technology Search	Writing Forum	Writing Search
DRAGON $w_{chunk}=1.0$	recall@3	0.6244	0.7126	0.5375	0.5779	0.2385	0.3695	0.3239	0.4362	0.6190	0.6471
	recall@10	0.7627	0.8636	0.6813	0.7359	0.3738	0.5462	0.5010	0.6023	0.7620	0.7871
DRAGON $w_{query}=1.0$	recall@3	0.6583	0.7247	0.5839	0.6071	0.2707	0.3647	0.3892	0.4866	0.6235	0.6583
	recall@10	0.8017	0.8654	0.7108	0.7478	0.3991	0.5219	0.5704	0.6812	0.7420	0.7656
DRAGON $w_{title}=1.0$	recall@3	0.6913	0.7610	0.6294	0.6472	0.3565	0.4408	0.4616	0.5436	0.6550	0.6928
	recall@10	0.8167	0.8790	0.7458	0.7879	0.4834	0.5948	0.6477	0.7064	0.7690	0.8011
DRAGON*	recall@3	0.6883	0.7625	0.6079	0.6472	0.3099	0.4498	0.4192	0.5285	0.6520	0.7031
	recall@10	0.8172	0.8911	0.7468	0.7944	0.4427	0.6062	0.6038	0.7097	0.7725	0.8170

BEIR Dataset								
Model	Recall	ArguAna	FIQA	Quora	SciDocs	SciFact	CQADupstack English	CQADupstack Physics
DRAGON $w_{chunk}=1.0$	recall@3	0.3919	0.3681	0.8587	0.2860	0.6601	0.4331	0.3638
	recall@10	0.6863	0.5196	0.9478	0.4700	0.7827	0.5338	0.4966
DRAGON $w_{query}=1.0$	recall@3	0.3265	0.3875	0.8267	0.2820	0.6032	0.4318	0.3503
	recall@10	0.6472	0.5220	0.9283	0.4470	0.7403	0.5344	0.4889
DRAGON $w_{title}=1.0$	recall@3	0.3208	0.4310	0.8039	0.2940	0.6375	0.4516	0.4081
	recall@10	0.6230	0.5692	0.9139	0.4770	0.7556	0.5567	0.5274
DRAGON*	recall@3	0.3663	0.4255	0.8638	0.3040	0.6610	0.4618	0.3936
	recall@10	0.6764	0.5635	0.9527	0.4800	0.7710	0.5662	0.5342

Table 5: Ablation study on doc-level embedding with DRAGON. In most cases the ensemble of relevant queries, title and chunks gives the best results. * means base model plus the doc-level embedding (chunk:0.3, query:0.6, title:0.3).

LoTTE Dataset											
Model	Recall	Lifestyle Forum	Lifestyle Search	Recreation Forum	Recreation Search	Science Forum	Science Search	Technology Forum	Technology Search	Writing Forum	Writing Search
ColBERTv2 chunk64 only	recall@3	0.6184	0.7474	0.5949	0.6245	0.3535	0.4797	0.3927	0.5168	0.6680	0.7199
	recall@10	0.7537	0.8759	0.7258	0.7586	0.4819	0.6353	0.5758	0.6846	0.7945	0.8273
ColBERTv2 query only	recall@3	0.7088	0.7413	0.6479	0.6580	0.3634	0.4327	0.3643	0.4530	0.6835	0.7274
	recall@10	0.8222	0.8759	0.7642	0.7727	0.4948	0.5997	0.5259	0.5419	0.7890	0.8254
ColBERTv2 title only	recall@3	0.6004	0.6218	0.5210	0.5487	0.3128	0.3695	0.4336	0.4715	0.5425	0.5780
	recall@10	0.7368	0.7458	0.6479	0.6937	0.4378	0.5024	0.5968	0.6141	0.6505	0.6853
ColBERTv2*	recall@3	0.7308	0.8003	0.6753	0.7100	0.4026	0.5024	0.4626	0.5956	0.7145	0.7544
	recall@10	0.8447	0.9107	0.7862	0.8268	0.5558	0.6726	0.6517	0.7383	0.8260	0.8571

BEIR Dataset								
Model	Recall	ArguAna	FIQA	Quora	SciDocs	SciFact	CQADupstack English	CQADupstack Physics
ColBERTv2 query only	recall@3	0.3122	0.4299	0.8037	0.2680	0.6041	0.4503	0.4187
	recall@10	0.5711	0.5654	0.9102	0.4170	0.7214	0.5357	0.5342
ColBERTv2 title only	recall@3	0.2091	0.3372	0.7149	0.2580	0.4806	0.3344	0.3494
	recall@10	0.3947	0.4588	0.8265	0.4060	0.6005	0.4248	0.4716
ColBERTv2*	recall@3	0.3592	0.4666	0.9067	0.3000	0.6862	0.4822	0.4196
	recall@10	0.6344	0.6018	0.9663	0.4850	0.7917	0.5694	0.5611

Table 6: Ablation study on doc-level embedding with ColBERTv2. In all cases the ensemble of relevant queries, title and chunks gives the best results. * means base model plus the doc-level embedding.

Question Set	#Questions	#Passages
ArguAna	1406	8674
FIQA	6648	57600
Quora	15000	522929
SciDocs	1000	25313
SciFact	1109	5183
CQADupstack English	1570	40221
CQADupstack Physics	1039	38316

Table 7: Statistical description of BEIR dataset

Question Set	#Questions	#Passages	Subtopics
Lifestyle Search	661	219k	Cooking, Sports, Travel
Lifestyle Forum	2002		
Recreation Search	924	167k	Gaming, Anime, Movies
Recreation Forum	2002		
Science Search	617	1.694M	Math, Physics, Biology
Science Forum	2017		
Technology Search	596	639k	Apple, Android, UNIX, Security
Technology Forum	2004		
Writing Search	1071	200k	English
Writing Forum	2000		

Table 8: Statistical description of LoTTE Test dataset

I will give you an article below. What are some search queries or questions that are relevant for this article or this article can answer?

Separate each query in a new line.

This is the article: {document}

Only provide the user queries without any additional text. Format every query as 'query:' followed by the question. Don't write empty queries.

Table 9: Prompt for generating relevant queries for documents

I will give you an article below. Create a title for the below article.

This is the article: {document}

Only provide the title without any additional text. Format the reply starting with 'title:' followed by the question. Don't write empty title.

Table 10: Prompt for generating titles for documents.